🚀 Collections Framework Masterclass

Chapter 12 — Deque Interface (Complete Deep Dive)

> **Interview Importance:** ⭐⭐⭐⭐⭐

`Deque` is one of the most useful interfaces in the Collections Framework because it can act as **both a Queue and a Stack**.

The name comes from:

> **Double Ended Queue**

It allows insertion and removal from **both ends**.

---

1. What is Deque?

Package:

```
java.util.Deque
```

Hierarchy:

```
Iterable
   ↓
Collection
   ↓
Queue
   ↓
Deque
```

Important implementations:

```
Deque
 ├── ArrayDeque
 └── LinkedList
```

For most normal single-threaded use cases, **ArrayDeque is the preferred implementation**.

---

2. Core Idea

A normal Queue:

```
Insert → Rear
Remove → Front
```

A Deque:

```
          Front
            ↓
     A ⇄ B ⇄ C ⇄ D
            ↑
          Rear

Insert/remove from either end
```

Therefore:

```
Deque
 ↓
Queue behavior
+
Stack behavior
```

---

3. Deque as a Queue

Use:

```
Deque<Integer> deque =
    new ArrayDeque<>();

deque.offerLast(10);
deque.offerLast(20);
deque.offerLast(30);
```

Then:

```
deque.pollFirst();
```

returns:

```
10
```

So:

```
offerLast() → insertion at rear
pollFirst()  → removal from front
```

This gives normal FIFO behavior.

---

4. Deque as a Stack

Use:

```
Deque<Integer> stack =
    new ArrayDeque<>();

stack.push(10);
stack.push(20);
stack.push(30);
```

Conceptually:

```
30 ← top
20
10
```

Then:

```
stack.pop();
```

returns:

```
30
```

This gives LIFO behavior.

---

5. Why Deque is Better Than Stack

Legacy:

```
Stack<Integer> stack =
    new Stack<>();
```

Modern:

```
Deque<Integer> stack =
    new ArrayDeque<>();
```

Why prefer Deque?

- `Stack` inherits from legacy `Vector`
- `Vector` synchronizes operations
- `ArrayDeque` avoids unnecessary synchronization overhead
- Deque provides a cleaner abstraction for stack operations

Therefore:

> **Use `Deque` when you need stack semantics in modern Java.**

---

6. Deque Method Families

This is the most important part to memorize.

First End

```
addFirst()
offerFirst()
removeFirst()
pollFirst()
```

```
getFirst()
peekFirst()
```

Last End

```
addLast()
offerLast()
removeLast()
pollLast()
getLast()
peekLast()
```

---

7. Method Pairs

Just like Queue:

```
Insertion

addFirst()  → exception on failure
offerFirst() → special-value failure

addLast()   → exception on failure
offerLast() → special-value failure
```

Removal:

```
removeFirst() → exception if empty
pollFirst()   → null if empty

removeLast()  → exception if empty
pollLast()    → null if empty
```

Inspection:

```
getFirst() → exception if empty
peekFirst() → null if empty

getLast() → exception if empty
peekLast() → null if empty
```

8. Easy Mental Model

```
          FRONT
            ↓
     +------+------+------+------+
     |  10  |  20  |  30  |  40  |
     +------+------+------+------+
                              ↑
                            REAR
```

Operations:

```
addFirst()  → 5 10 20 30 40

addLast()   → 10 20 30 40 50

pollFirst() → removes 10

pollLast()  → removes 40
```

9. ArrayDeque

`ArrayDeque` is the most important Deque implementation.

Package:

```
java.util.ArrayDeque
```

Declaration:

```
public class ArrayDeque<E>
    extends AbstractCollection<E>
    implements Deque<E>,
               Cloneable,
               Serializable
```

It is:

- Resizable
- Array-backed
- Not synchronized
- Does not permit `null`
- Efficient at both ends

---

10. ArrayDeque Internal Structure

Unlike LinkedList, ArrayDeque does not create a Node object for every element.

It uses an internal array.

Conceptually:

```
+----+----+----+----+----+----+----+
|    | A  | B  | C  | D  |    |    |
+----+----+----+----+----+----+----+
     ↑                      ↑
   front                  rear
```

It maintains positions indicating where elements should be inserted/removed.

---

11. Why ArrayDeque Is Efficient

Adding/removing from either end generally requires only adjusting internal positions.

There is no need to:

```
Shift all elements
```

as would happen with inserting/removing at the beginning of an ArrayList.

Typical complexity:

```
addFirst()   → O(1) amortized
addLast()    → O(1) amortized
```

```
pollFirst()  → O(1)
pollLast()   → O(1)

peekFirst()  → O(1)
peekLast()   → O(1)
```

---

12. Circular Buffer Concept

A useful mental model for ArrayDeque is a **circular array**.

Suppose the array reaches its end:

```
[A][B][C][D][ ][ ]
```

The next insertion doesn't necessarily need to move everything.

The logical positions can wrap around:

```
[ ][ ][C][D][A][B]
```

The implementation uses modular/wraparound indexing to treat the array as circular.

When the array becomes full, it grows.

---

13. Why Circular Structure?

Without circular indexing, repeatedly removing from the front could create unused space.

For example:

```
Before:

[A][B][C][D][ ][ ]

Remove A, B:

[ ][ ][C][D][ ][ ]
```

Instead of shifting `C` and `D`, ArrayDeque can move its logical front position.

This keeps end operations efficient.

---

14. ArrayDeque Growth

ArrayDeque is dynamically resizable.

When there is insufficient space:

```
Current array
      ↓
Create larger array
      ↓
Copy elements
      ↓
Continue operation
```

Therefore occasional resizing costs O(n), while ordinary insertion remains amortized O(1).

The exact growth policy is implementation-specific and should not be confused with ArrayList's approximately 1.5× rule.

---

15. Why Doesn't ArrayDeque Allow null?

Example:

```
Deque<String> deque =
    new ArrayDeque<>();

deque.offer(null);
```

This throws:

```
NullPointerException
```

Why?

Because Deque methods such as:

```
peek()
poll()
```

use `null` to represent:

```
No element available
```

Allowing null would make the result ambiguous.

---

16. ArrayDeque vs LinkedList

Both implement Deque.

But their internal structures differ:

```
ArrayDeque
→ resizable array

LinkedList
→ doubly linked nodes
```

Comparison:

Feature	ArrayDeque	LinkedList
Structure	Array-based	Doubly linked
End operations	O(1) amortized	O(1)
Memory overhead	Lower	Higher
Cache locality	Better	Worse
Null	❌	✅
Thread-safe	❌	❌
Typical Deque choice	✅	Usually not

For a pure Queue/Deque workload, prefer `ArrayDeque` unless you have a specific reason to use `LinkedList`.

---

17. Deque vs Stack

Feature	Deque	Stack
Modern API	✅	❌ Legacy
Queue behavior	✅	❌
Stack behavior	✅	✅
Double-ended operations	✅	❌
Synchronization	Depends on implementation	Vector-based synchronization
Preferred for new code	✅	❌

---

18. Stack Operations Using Deque

Use:

```
Deque<Integer> stack =
    new ArrayDeque<>();
```

Operations:

```
push() → addFirst()
pop()  → removeFirst()
peek() → peekFirst()
```

This gives:

```
LIFO
```

---

19. Queue Operations Using Deque

For FIFO:

```
offerLast()
pollFirst()
peekFirst()
```

Example:

```
Deque<Integer> queue =
    new ArrayDeque<>();

queue.offerLast(10);
queue.offerLast(20);
queue.offerLast(30);

System.out.println(
    queue.pollFirst()
);
```

Output:

```
10
```

---

20. Deque as Both

One Deque can technically support either behavior.



This flexibility is one of its biggest advantages.

---

21. Descending Iterator

Deque provides:

```
descendingIterator()
```

Example:

```
Deque<Integer> deque =
    new ArrayDeque<>();

deque.addLast(10);
deque.addLast(20);
deque.addLast(30);

Iterator<Integer> it =
    deque.descendingIterator();
```

Iteration:

```
30
20
10
```

Useful when you need reverse traversal.

---

22. `removeFirstOccurrence()`

Deque also provides:

```
removeFirstOccurrence(Object o)
```

Example:

```
deque.removeFirstOccurrence(20);
```

It removes the first matching occurrence.

---

23. `removeLastOccurrence()`

Similarly:

```
deque.removeLastOccurrence(20);
```

removes the last matching occurrence.

These operations require searching and therefore are generally:

```
O(n)
```

---

24. Deque Does Not Provide Random Access

You cannot do:

```
deque.get(3);
```

like an ArrayList.

Deque is designed around:

```
Front
+
Rear
```

not arbitrary indexes.

---

25. Time Complexity

For ArrayDeque:

Operation	Complexity
`addFirst()`	O(1) amortized
`addLast()`	O(1) amortized

Operation	Complexity
`offerFirst()`	O(1) amortized
`offerLast()`	O(1) amortized
`removeFirst()`	O(1)
`removeLast()`	O(1)
`pollFirst()`	O(1)
`pollLast()`	O(1)
`peekFirst()`	O(1)
`peekLast()`	O(1)
`contains()`	O(n)
`remove(Object)`	O(n)
Iteration	O(n)

---

26. Real-World Use Cases

Stack

Expression evaluation:

```
Infix → Postfix
```

Balanced parentheses:

```
( [ { } ] )
```

Undo operations:

```
Action 1
Action 2
Action 3
       ↓
     Stack
```

Queue

Task processing:

```
Request
  ↓
Deque
  ↓
Worker
```

Sliding Window Algorithms

Deque is extremely useful for problems such as:

> Find the maximum value in every window of size K.

A monotonic deque can solve this efficiently.

This is a common coding-interview application.

Palindrome Checking

You can compare:

```
front ↔ rear
```

using:

```
pollFirst()
pollLast()
```

---

27. ⭐ Interview Scenario

Requirement

Implement a stack without using `Stack`.

Answer:

```
Deque<Integer> stack =
    new ArrayDeque<>();
```

Then:

```
stack.push(10);
stack.push(20);
stack.push(30);

stack.pop();
```

Result:

```
30
```

---

28. ⭐ Interview Scenario

Requirement

Implement a FIFO Queue using Deque.

Answer:

```
Deque<Integer> queue =
    new ArrayDeque<>();

queue.offerLast(10);
queue.offerLast(20);

queue.pollFirst();
```

Result:

```
10
```

---

29. ⭐ Interview Scenario

Requirement

You need to insert/remove from both ends efficiently.

Example:

```
Front ↔ Rear
```

Use:

```
Deque<Integer> deque =
    new ArrayDeque<>();
```

---

30. Common Interview Traps

Trap 1

> Deque is only a Queue.

❌ Wrong.

It supports both Queue and Stack semantics.

Trap 2

> ArrayDeque allows null.

❌ Wrong.

It rejects null.

Trap 3

> ArrayDeque is thread-safe.

❌ Wrong.

It is not synchronized.

Trap 4

> LinkedList is always better for Deque because insertion/removal is O(1).

Incomplete.

ArrayDeque also provides efficient end operations and usually has better memory locality and lower per-element overhead.

Trap 5

> ArrayDeque grows exactly like ArrayList.

❌ Wrong.

Don't assume ArrayList's ~1.5× growth formula applies to ArrayDeque.

---

31. Frequently Asked Interview Questions

Q1. What is Deque?

A double-ended queue that allows insertion and removal from both ends.

Q2. Can Deque act as a Stack?

Yes.

Use:

```
push()
pop()
peek()
```

Q3. Can Deque act as a Queue?

Yes.

For example:

```
offerLast()
pollFirst()
peekFirst()
```

Q4. Why is ArrayDeque preferred over Stack?

Because Stack is a legacy Vector-based class with synchronization overhead, while ArrayDeque is designed for efficient deque/stack operations.

Q5. Why doesn't ArrayDeque allow null?

Because `null` is used as the special return value by methods such as `poll()` and `peek()` when the deque is empty.

Q6. Is ArrayDeque thread-safe?

No.

Q7. ArrayDeque vs LinkedList?

ArrayDeque is generally preferred for pure Queue/Deque workloads because it is array-backed, has lower per-element overhead, and typically has better cache locality.

Q8. What is the complexity of addFirst/addLast?

O(1) amortized.

Q9. Can ArrayDeque perform random access?

No.

Q10. What is `descendingIterator()`?

It returns an iterator that traverses the Deque from rear to front.

---

32. ⭐ Coding Interview Pattern

Problem

Reverse a sequence using Deque.

```
Deque<Integer> deque =
    new ArrayDeque<>();

deque.addLast(10);
deque.addLast(20);
deque.addLast(30);

while (!deque.isEmpty()) {
    System.out.println(
        deque.pollLast()
    );
}
```

Output:

```
30
20
10
```

The key operation is:

```
pollLast()
```

---

33. Decision Framework

```
Need double-ended operations?
            ↓
          Deque
            ↓
      ArrayDeque
```

Need Stack?

```
Deque
 ↓
ArrayDeque
 ↓
push/pop/peek
```

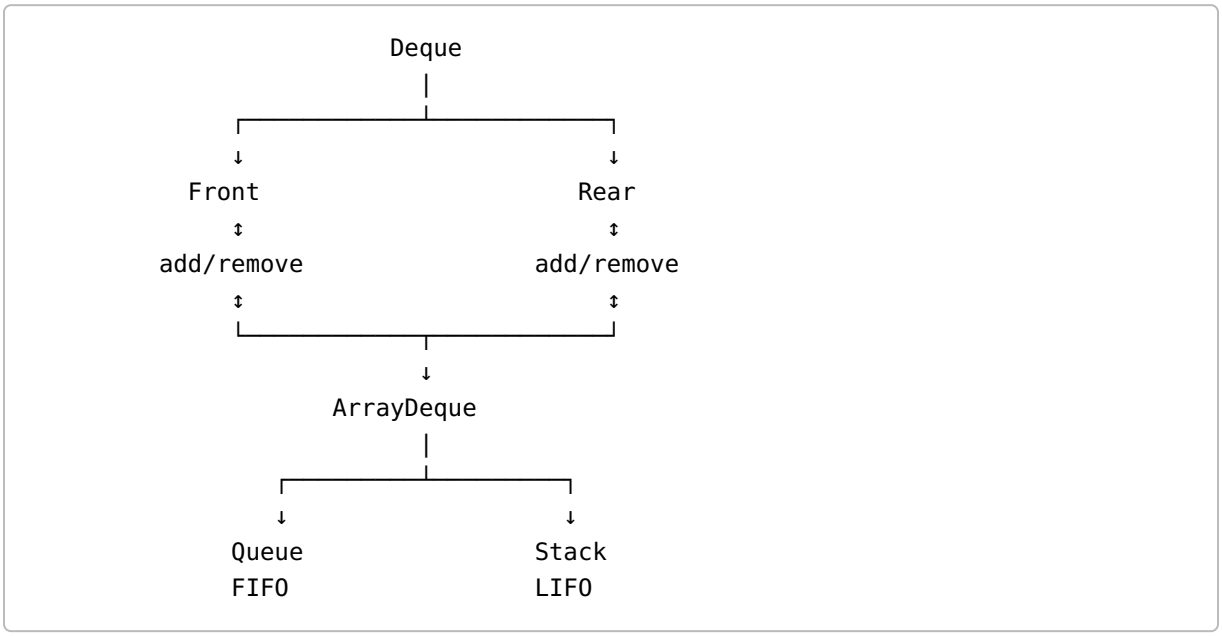
Need FIFO Queue?

```
Deque
 ↓
ArrayDeque
 ↓
offerLast/pollFirst
```

Need concurrent producer-consumer?

```
Don't use ArrayDeque
        ↓
BlockingQueue
```

🧠 Final Mental Model



The most important mapping:

```
Queue:
offerLast()
pollFirst()
peekFirst()

Stack:
push()
pop()
peek()
```

🔥 Chapter Summary

Concept	Key Point
Deque	Double-ended Queue
Insert	Both ends
Remove	Both ends
Queue behavior	FIFO
Stack behavior	LIFO
Preferred implementation	ArrayDeque
ArrayDeque structure	Resizable array
End operations	O(1) amortized
Random access	No
Null	Not allowed
Thread-safe	No
Reverse traversal	`descendingIterator()`
Main advantage	Efficient operations at both ends

🎯 Interview Readiness Checklist

You should be able to answer:

- [ ] What is Deque?
- [ ] Why is it called a double-ended Queue?

- [ ] How can Deque implement a Stack?
- [ ] How can Deque implement a Queue?
- [ ] `addFirst()` vs `offerFirst()`?
- [ ] `removeFirst()` vs `pollFirst()`?
- [ ] `getFirst()` vs `peekFirst()`?
- [ ] Why is ArrayDeque preferred over Stack?
- [ ] ArrayDeque vs LinkedList?
- [ ] Why doesn't ArrayDeque allow null?
- [ ] Is ArrayDeque thread-safe?
- [ ] What is the circular-array concept?
- [ ] What is `descendingIterator()`?
- [ ] What are `removeFirstOccurrence()` and `removeLastOccurrence()`?
- [ ] What is the complexity of operations at both ends?

🚀 Next Chapter

Chapter 13 — PriorityQueue (Complete Deep Dive)

We'll focus on what's new:

- Why PriorityQueue is NOT FIFO
- Binary Heap internals
- Min-Heap vs Max-Heap
- `Comparable` vs `Comparator`
- Parent/child index calculations
- `offer()` and heap insertion
- `poll()` and heap removal
- Sift-up and sift-down
- Why `peek()` is O(1)
- Why `poll()` is O(log n)
- PriorityQueue ordering surprises
- Custom objects
- Real-world scheduling use cases
- Top-K problems
- 20+ interview questions